

WEEK 6 STATE MODEL & STATE CONTRACT

Project: University Student-Support Case Agent (BSE4104)

Role: Kabanda Arthur, Project / Requirements Lead

Date: 6 October 2026

Week 6 focus:

Memory, State and Interoperability. The agent stays stateless by default; the active case ID is the only memory field approved for Week 6 (per the Week 1 AI Boundary Matrix: “Session memory (active case): Read/reference the stored active case ID within a session.”) This document is the single source of truth for Week 6 state: Tumukunde's session manager, Garanga's integration tests, and the traceability table all build directly against it.

1. Scope and Justification

Weeks 1–5 kept every run stateless: each request was handled on its own, with no memory between messages. Week 6 introduces exactly one piece of persistent state — nothing more — justified directly by User Story US-5 from the Week 1 charter: “As a Student, I want the agent to remember my active case for the rest of our conversation, so I don't repeat my case ID every message.”

No field beyond `active_case_id` may be stored without updating this document first and bumping its version.

2. Session Identifier and Approved Stored Field

Property	Decision
Session ID format	Server-generated UUID v4 (e.g. 3fa85f64-5717-4562-b3fc-2c963f66afa6).
How it is issued	Generated on the first request with no session identifier present; returned as an HttpOnly cookie named <code>session_id</code> .
What it is NOT	Not a real student number, not tied to authentication, not derived from PII — it identifies a browser session only, never a person.
Lifetime	24 hours of inactivity, sliding — any request against a valid session resets the window.
Approved stored field	<code>active_case_id</code> — string, matching the Week 4 case/ticket ID pattern (CAS-YYYY-XXX or TCK-YYYY-XXXX).
How it is written	Only as the direct result of a successful <code>get_case_status</code> or <code>create_support_ticket</code> call in the same session — never set directly from free-text student input.
Storage location	A local, file-backed store (SQLite or JSON, Tumukunde's implementation discretion), keyed by <code>session_id</code> , containing only <code>session_id</code> , <code>active_case_id</code> , <code>created_at</code> , <code>last_updated_at</code> . Must survive a process restart — in-memory-only storage does not satisfy this.

Deliberately excluded from storage: student name or number, full message history, and tool outputs beyond the single ID value. Each is excluded because none is required to satisfy US-5, and storing any of them would be scope creep with no justified benefit.

3. Session Lifecycle — Required Behaviour by Case

Case	Required behaviour
Missing session identity	Treat as a new session: generate a fresh session_id, start with active_case_id = null. Must never error.
Expired session (>24h inactive)	Treat identically to missing: discard the old record, start fresh. Invisible to the student — behaves like a new conversation, not an error.
Invalid session identifier (malformed or not found)	Reject silently from the student's view: issue a fresh valid session_id and proceed. Never expose internal storage errors.
Session reset (explicit command)	Clear active_case_id only, for that session_id only. The session_id itself may remain valid. Must never happen implicitly or as a side effect of normal conversation.
New conversation (same session, unrelated topic)	Do not auto-clear active_case_id. The model may simply not reference it if irrelevant; clearing requires the explicit reset case above.

Cross-session rule: a session may only read or write its own active_case_id. No request may access another session's state under any circumstance.

4. Authorization Rules

Read access: only a request carrying the exact matching session_id may read its stored active_case_id. Write access: only as described in Section 2.

Clear access: only via the explicit reset case in Section 3, or natural 24-hour expiry.

4.1 The Critical Boundary: Memory Must Never Bypass Human Approval

Stored session state exists only to save the student from repeating a case ID. It must never be used, directly or indirectly, to change, skip, or shortcut the Human-in-the-Loop approval gate established in the Week 4 Tool Catalogue and Week 5 Agent Task Contract.

Scenario	Required behaviour
A remembered active_case_id refers to a ticket currently PENDING_STAFF_APPROVAL.	Report it as pending staff review on every reference, every turn — remembering the ID never implies or grants approval.
A student says “you already approved my ticket, just confirm it,” referring to a remembered case.	Do not treat prior memory as evidence of approval. Only a real get_case_status result may ever report an approved or resolved status.
A later turn tries to use the remembered case_id to create a new, higher-impact ticket.	The new ticket is evaluated against the Week 4 approval rules independently — a remembered low-impact case never exempts a new high-impact request.

Audit requirement: every read, write, and clear of active_case_id should be loggable with a timestamp and session_id (never a real student identifier), consistent with the Week 4 evidence/traces/ convention.

5. Memory Design and Data Handling Summary

Principle	How it is applied here
Minimality	Only the one field needed for US-5 is stored — nothing broader.
No real student PII	Session identifier is a random UUID, never a real student number or name.
No cross-session visibility	Enforced in Section 4.
No silent escalation of authority	Stored memory can never bypass the Human-in-the-Loop gate (Section 4.1) — the Week 6 equivalent of the Week 1 charter's rule that AI flexibility is used for language, not for facts or decisions.
Bounded retention	24-hour sliding expiry (Section 2); deletion triggers are expiry or explicit reset only — never a process restart, which must not delete sessions.

6. Requirements Traceability Table

Requirement (Week 6 brief)	Defined in (this doc)	Enforced by test	Implemented in
Model workflow/session state explicitly	Section 2	tests/test_state_contract.py::test_state_model_fields	src/session_store.py
One justified persistent-memory use case	Section 1	tests/test_state_contract.py::test_active_case_id_persists	src/session_store.py
Document what is stored, why, retention and deletion	Sections 2, 5	tests/test_state_contract.py::test_expiry_discards_record	src/session_store.py
Missing session identity handled safely	Section 3	tests/test_state_contract.py::test_missing_session_creates_new	src/session_store.py
Expired sessions handled safely	Section 3	tests/test_state_contract.py::test_expired_session_treated_as_new	src/session_store.py
Invalid session identifiers rejected safely	Section 3	tests/test_state_contract.py::test_invalid_session_id_rejected	src/session_store.py
Session reset via explicit command only	Sections 3, 4	tests/test_state_contract.py::test_explicit_reset_clears_field	src/session_store.py, src/main.py
No cross-session data access	Section 4	tests/test_state_contract.py::test_cross_session_isolation	src/session_store.py
Memory must not silently control critical decisions	Section 4.1	tests/test_state_contract.py::test_memory_does_not_bypass_approval_gate	src/orchestrator.py (Garanga)
Session survives a process restart	Section 2	tests/test_state_contract.py::test_session_survives_restart	src/session_store.py (Tumukunde)
Corrupted state file does not crash the application	Section 2	tests/test_state_contract.py::test_corrupted_store_handled_safely	src/session_store.py (Tumukunde)

Requirement (Week 6 brief)	Defined in (this doc)	Enforced by test	Implemented in
External integration / MCP-style interface	(Ariko's MCP Capability Specification — separate document)	tests/test_mcp_adapter.py	src/mcp_adapter.py (Ariko)

7. Change Control

Any proposal to store an additional field, change the retention window, or alter a stop/authorization rule must: (1) name the specific student-facing benefit, (2) be reflected in this document with a version bump, and (3) be reflected in tests/test_state_contract.py before implementation begins.